



MÓDULO BÔNUS • AULA B4

Bônus 4

Abertura e documentação de bugs

Um bug bem descrito é meio caminho para a correção.

Instrutor: Douglas Adriano Queiroz

O que vamos ver hoje



Anatomia do bug

O que todo report tem



Severidade × prioridade

Não é a mesma coisa



Níveis de severidade

Do leve ao crítico



Evidências

Print, vídeo e logs



Ciclo de vida

Do aberto ao fechado



Boas práticas

Como descrever bem

A anatomia de um bom relatório de bug

Bug mal descrito volta para você ('não consegui reproduzir'). Bug bem descrito é corrigido rápido.

Título objetivo: 'Vaga com valor negativo é salva sem erro' (não 'deu errado').

Passos para reproduzir: 1, 2, 3 — qualquer pessoa repete e vê o problema.

Esperado × obtido: o que deveria acontecer e o que de fato aconteceu.

Ambiente: navegador/aparelho, versão, usuário, data/hora.



Analogia

É a diferença entre dizer ao mecânico 'o carro faz barulho' e 'há um clique ao virar à esquerda acima de 40 km/h'. Quanto melhor a descrição, mais rápido o conserto — e sem o carro voltar pra você.



Leve daqui: Reprodução clara + esperado × obtido + ambiente = bug que se resolve.

Severidade × Prioridade



Severidade (impacto)

- O quanto o defeito ESTRAGA o sistema.
- Definida pela técnica (QA/dev).
- Ex.: o app fecha sozinho = alta severidade.



Prioridade (urgência)

- O quão RÁPIDO precisa ser corrigido.
- Definida pelo negócio (PO/cliente).
- Ex.: erro de logo na home = corrigir já (alta prioridade).



Leve daqui: Severidade = tamanho do estrago; prioridade = pressa para corrigir. Podem divergir.

Exemplos: severidade e prioridade juntas



Alta sev. + Alta prio.

App de pagamento aceita cobrança errada: estraga muito E precisa de correção imediata.



Alta sev. + Baixa prio.

Função rara que trava, usada por pouca gente: grave, mas pode esperar.



Baixa sev. + Alta prio.

Nome da empresa escrito errado na home: pequeno, mas embaraçoso — corrigir já.



Baixa sev. + Baixa prio.

Espaçamento torto numa tela interna: pode entrar na fila tranquilamente.

Evidências: mostre, não só conte

Evidência transforma 'acho que tem um bug' em 'aqui está o bug'.

Print de tela: destaque o erro com um círculo/seta.

Vídeo / GIF: ideal para bugs de fluxo e de tempo.

Logs e console: a mensagem de erro técnica (F12 → Console/Network).

Dados usados: qual vaga, qual usuário, quais valores reproduzem.



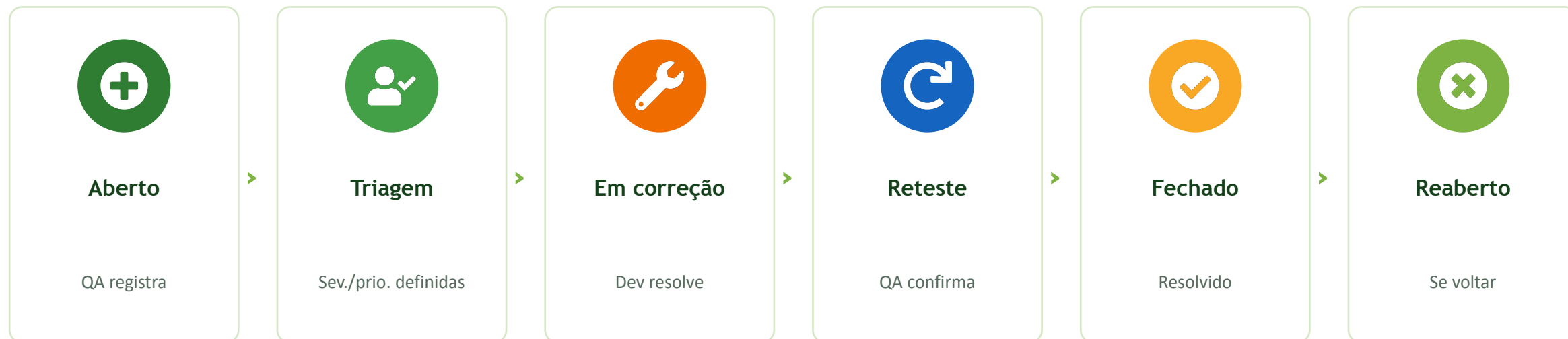
Analógia da casa

É a história com foto: você não diz só 'tem infiltração'; você fotografa a mancha, marca o cômodo e anota o dia de chuva. Com a prova, ninguém duvida nem demora a agir.



Leve daqui: Sem evidência, o bug vira discussão; com evidência, vira tarefa.

O ciclo de vida de um bug



Boas práticas na descrição

Um problema por relatório (não junte vários). Título específico e pesquisável. Seja factual, sem culpar ninguém ('o sistema faz X', não 'o dev errou'). Inclua sempre passos + evidência. No FreelaBairro: 'CT-12 — Vaga aceita valor negativo': passos, esperado (recusar com mensagem), obtido (salva R\$ -50), print e o registro no banco.



Resumão da aula



Bug report = título objetivo, passos, esperado × obtido e ambiente.



Severidade = impacto técnico; Prioridade = urgência do negócio.



Severidade e prioridade podem divergir (ex.: logo errado na home).



Evidências (print, vídeo, logs, dados) tornam o bug inegável.



Ciclo do bug: aberto → triagem → correção → reteste → fechado.



Boas práticas: um bug por report, factual e sempre com evidência.



Glossário da aula

Bug / defeito — Problema no software que causa comportamento errado

Relatório de bug — Registro estruturado de um defeito

Reproduzir — Refazer os passos que causam o bug

Esperado × obtido — O que deveria acontecer × o que aconteceu

Severidade — Quanto o defeito impacta o sistema (técnico)

Prioridade — Quão urgente é corrigir (negócio)

Evidência — Print, vídeo, log ou dado que comprova o bug

Triagem — Análise inicial que define severidade e prioridade

Ciclo de vida do bug — Estados do bug, do aberto ao fechado

Reteste — Confirmar que a correção resolveu o defeito



Referências e para aprofundar

ISTQB CTFL — Cap. 5. Gestão de defeitos e relatórios de teste.

Atlassian / Jira. Boas práticas de registro e triagem de bugs.

Projeto do curso: exemplos aplicados ao app FreelaBairro — vagas de freelancer por bairro (Floripa / Jaraguá do Sul).